VOLCANO-CA-MC: DOKUMENTACJA TECHNICZNA

Zestawienie wyników: przykładowe przypadki (wulkan: CA + MC)

Bartosz Pajda

Wygenerowano: 22.10.2026

Wygenerowane: 15.10.2026 03:52 skryptem `tools/zestaw_wynikow.py` · procesor: 13th Gen Intel(R) Core(TM) i9-13900HX · kompilator: MSVC 19.38 Release

Każdy punkt: co sprawdzamy, polecenie (z folderu `volcano-ca-mc`), wynik odczytany z plików silnika (`times.json`, `stats.json`). Wyniki są w `results\<nazwa>\` i można je obejrzeć w GUI (Podgląd 2D / 3D, Czasy i statystyki).

> Czasy pochodzą z jednego uruchomienia (MSVC Release) i służą orientacji. Pomiary do profilu (3 powtórzenia, przypięcie do rdzenia P) są w `docs/profilowanie.md`, porównanie CA vs MC w `docs/kolejka-serie-dane.md`.

#	Przypadek	Status
1	Etna: spływ lawy na prawdziwym terenie (CA, 3D)	✅ OK (kod 0, 21,5 s z uruchomieniem procesu)
2	Wezuwiusz: wypełnienie krateru i przelew (CA, 3D)	✅ OK (kod 0, 4,5 s z uruchomieniem procesu)
3	Przekrój 2D: reguły automatu	✅ OK (kod 0, 0,1 s z uruchomieniem procesu)
4	Klasyczny rozrost ziaren CA (100³, 10 ziaren)	✅ OK (kod 0, 0,7 s z uruchomieniem procesu)
5	Lawa 2D + krystalizacja bazaltu MC (z migawkami MC)	✅ OK (kod 0, 0,1 s z uruchomieniem procesu)
6	Lawa 3D + krystalizacja bazaltu MC	✅ OK (kod 0, 1,6 s z uruchomieniem procesu)
7	Klasyczny naiwny Monte Carlo (200×200, Q = 50)	✅ OK (kod 0, 0,2 s z uruchomieniem procesu)
8	Teren z pliku: kaldera z wyłomem	✅ OK (kod 0, 3,1 s z uruchomieniem procesu)
9	Walidacja configu	✅
10	Benchmark vector vs macierz	✅

1. Etna: spływ lawy na prawdziwym terenie (CA, 3D)

Co sprawdzamy: Rozrost CA (lawa = ziarna z 3 kraterów szczytowych) na mapie wysokości z AWS Terrain Tiles; kratery w zadanych kolumnach (`vents.positions`).

Polecenie: (podgląd w GUI: wynik `etna_real`)

```
.\build\Release\engine.exe --config configs\etna_real.json --out results\etna_real
```

Status: ✅ OK (kod 0, 21,5 s z uruchomieniem procesu)

	Wynik
siatka	160 × 160 × 70 = 1 792 000 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	284 kroków, brak płynnej lawy
objętość lawy	4 402 komórek (płynnej na końcu: 0), największy zasięg od krateru: 46,2 komórek ≈ 10,8 km
lawa z kraterów	krater 1: 292 kom., zasięg 8,5; krater 2: 1 250 kom., zasięg 43,0; krater 3: 2 860 kom., zasięg 46,2
czas CA (`caSim`)	21,39 s (42,0 ns na komórkę i krok)
pamięć (szczyt)	76,6 MiB zmierzone, model z kodu 71,0 MiB (41,5 B/komórkę)
pliki	`config_used.json` 1,1 KiB, `ca_result.bin` 22,2 MiB, `stats.json` 6,1 KiB
czas całkowity (`total`)	21,48 s

2. Wezuwiusz: wypełnienie krateru i przelew (CA, 3D)

Co sprawdzamy: Wypełnianie zagłębienia (priority-flood) i przelew przez najniższe miejsce krawędzi; Monte Somma zatrzymuje lawę.

Polecenie: (podgląd w GUI: wynik `wezuwiusz_real`)

```
.\build\Release\engine.exe --config configs\wezuwiusz_real.json --out results\wezuwiusz_real
```

Status: ✅ OK (kod 0, 4,5 s z uruchomieniem procesu)

	Wynik
siatka	140 × 140 × 50 = 980 000 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	128 kroków, brak zmian w kroku
objętość lawy	2 470 komórek (płynnej na końcu: 1), największy zasięg od krateru: 44,0 komórek ≈ 3,5 km
czas CA (`caSim`)	4,43 s (35,3 ns na komórkę i krok)
pamięć (szczyt)	44,8 MiB zmierzone, model z kodu 39,2 MiB (42,0 B/komórkę)

	Wynik
pliki	`config_used.json` 1,0 KiB, `ca_result.bin` 12,1 MiB, `stats.json` 2,8 KiB
czas całkowity (`total`)	4,45 s

3. Przekrój 2D: reguły automatu

Co sprawdzamy: Podstawowe reguły lawy w przekroju x–z: spadek, rozlewanie z podparciem, kierunek największego spadku, stygnięcie i zastyganie w bazalt.

Polecenie: (podgląd w GUI: wynik `test_2d`)

```
.\build\Release\engine.exe --config configs\test_2d.json --out results\test_2d
```

Status: ✅ OK (kod 0, 0,1 s z uruchomieniem procesu)

	Wynik
siatka	200 × 1 × 100 = 20 000 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	351 kroków, brak płynnej lawy
objętość lawy	99 komórek (płynnej na końcu: 0), największy zasięg od krateru: 63,0 komórek
czas CA (`caSim`)	107,7 ms (15,3 ns na komórkę i krok)
pamięć (szczyt)	7,7 MiB zmierzone, model z kodu 1,8 MiB (94,4 B/komórkę)
pliki	`config_used.json` 0,9 KiB, `ca_result.txt` 339,1 KiB, `ca_result.bin` 253,9 KiB, `stats.json` 6,3 KiB
czas całkowity (`total`)	114,7 ms

4. Klasyczny rozrost ziaren CA (100^3, 10 ziaren)

Co sprawdzamy: Czysty rozrost ziaren CA (tryb `classic`, von Neumann, brzeg periodyczny): przypadek referencyjny do porównań wydajności.

Polecenie: (podgląd w GUI: wynik `classic_ref`)

```
.\build\Release\engine.exe --config configs\classic_ref.json --out results\classic_ref
```

Status: ✅ OK (kod 0, 0,7 s z uruchomieniem procesu)

	Wynik
siatka	100 × 100 × 100 = 1 000 000 komórek, sąsiedztwo von_neumann, brzeg periodic
kroki CA / koniec	71 kroków, brak wolnych komórek (AIR)
ziarna	10 ziaren, średnio 100 000 komórek (najmniejsze 36 606, największe 147 879)
czas CA (`caSim`)	499,6 ms (7,0 ns na komórkę i krok)
pamięć (szczyt)	45,4 MiB zmierzone, model z kodu 38,7 MiB (40,6 B/komórkę)
pliki	`config_used.json` 1,0 KiB, `ca_result.txt` 17,0 MiB, `ca_result.bin` 12,4 MiB, `stats.json` 3,1 KiB
czas całkowity (`total`)	670,8 ms

5. Lawa 2D + krystalizacja bazaltu MC (z migawkami MC)

Co sprawdzamy: CA → bazalt → Monte Carlo (Potts, Metropolis) z mobilnością ze stygnięcia: duże kryształy w jeziorze lawy, drobne na szybko zastygłych jęzorach; migawki MC do odtwarzania wzrostu ziaren.

Polecenie: (podgląd w GUI: wynik `lava_mc_2d_migawki`)

```
.\build\Release\engine.exe --config configs\lava_mc_2d.json --out results\lava_mc_2d_migawki
--set mc.snapshotsEvery=10
```

Status: ✅ OK (kod 0, 0,1 s z uruchomieniem procesu)

	Wynik
siatka	240 × 1 × 80 = 19 200 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	202 kroków, brak zmian w kroku
objętość lawy	516 komórek (płynnej na końcu: 1), największy zasięg od krateru: 116,0 komórek

	Wynik
czas CA (`caSim`)	58,5 ms (15,1 ns na komórkę i krok)
MC: domena	basalt: 516 komórek, Q = 50, 300 MCS, kT = 0.0, mobilność: cooling
MC: ziarna	486 → 74; średnie ziarno 1,06 → 6,97 kom., największe 39
MC: energia / akceptacja	1 310 → 359; zaakceptowano 6,6 % z 105 339 prób
MC: efekt stygnięcia	wolno stygnące (mobilność ≥ 0.5): średnie ziarno 10,9 kom.; szybko stygnące: 3,4 kom. → **×3,2**
czas MC (`mcSim`)	8,2 ms (78,2 ns na próbę)
pamięć (szczyt)	7,7 MiB zmierzone, model z kodu 1,7 MiB (94,7 B/komórkę)
pliki	`config_used.json` 0,9 KiB, `ca_result.txt` 327,6 KiB, `ca_result.bin` 243,8 KiB, `mc_snapshots` 2,3 MiB, `mc_result.txt` 214,5 KiB, `mc_result.bin` 75,0 KiB, `mc_stats.csv` 1,0 KiB, `stats.json` 5,4 KiB
czas całkowity (`total`)	98,9 ms

6. Lawa 3D + krystalizacja bazaltu MC

Co sprawdzamy: To samo co wyżej w 3D (80×80×50, sąsiedztwo Moore'a).

Polecenie: (podgląd w GUI: wynik `lava_mc_3d`)

```
.\build\Release\engine.exe --config configs\lava_mc_3d.json --out results\lava_mc_3d
```

Status: ✅ OK (kod 0, 1,6 s z uruchomieniem procesu)

	Wynik
siatka	80 × 80 × 50 = 320 000 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	133 kroków, brak zmian w kroku
objętość lawy	4 291 komórek (płynnej na końcu: 1), największy zasięg od krateru: 43,3 komórek
czas CA (`caSim`)	1,32 s (31,1 ns na komórkę i krok)
MC: domena	basalt: 4 291 komórek, Q = 50, 200 MCS, kT = 0.0, mobilność: cooling
MC: ziarna	3 719 → 574; średnie ziarno 1,15 → 7,48 kom., największe 168
MC: energia / akceptacja	27 959 → 13 649; zaakceptowano 8,6 % z 364 908 prób
MC: efekt stygnięcia	wolno stygnące (mobilność ≥ 0.5): średnie ziarno 25,8 kom.; szybko stygnące: 5,2 kom. → **×5,0**
czas MC (`mcSim`)	94,3 ms (258,3 ns na próbę)
pamięć (szczyt)	20,1 MiB zmierzone, model z kodu 14,4 MiB (47,1 B/komórkę)
pliki	`config_used.json` 0,9 KiB, `ca_result.txt` 5,4 MiB, `ca_result.bin` 4,0 MiB, `mc_result.txt` 3,5 MiB, `mc_result.bin` 1,2 MiB, `mc_stats.csv` 0,8 KiB, `stats.json` 4,3 KiB
czas całkowity (`total`)	1,55 s

7. Klasyczny naiwny Monte Carlo (200×200, Q = 50)

Co sprawdzamy: Kroki 1–5 naiwnego algorytmu Monte Carlo na całej przestrzeni, bez CA: zgrubianie ziaren w czasie (MCS).

Polecenie: (podgląd w GUI: wynik `mc_full_2d`)

```
.\build\Release\engine.exe --config configs\mc_full_2d.json --out results\mc_full_2d
```

Status: ✅ OK (kod 0, 0,2 s z uruchomieniem procesu)

	Wynik
siatka	200 × 200 × 1 = 40 000 komórek, sąsiedztwo von_neumann, brzeg periodic
MC: domena	full: 40 000 komórek, Q = 50, 100 MCS, kT = 0.0, mobilność: off
MC: ziarna	38 327 → 5 596; średnie ziarno 1,04 → 7,15 kom., największe 44
MC: energia / akceptacja	78 326 → 33 431; zaakceptowano 13,0 % z 4 000 000 prób
czas MC (`mcSim`)	185,4 ms (46,3 ns na próbę)

	Wynik
pamięć (szczyt)	7,7 MiB zmierzone, model z kodu 2,0 MiB (51,7 B/komórkę)
pliki	`config_used.json` 0,9 KiB, `mc_result.txt` 497,1 KiB, `mc_result.bin` 156,3 KiB, `mc_stats.csv` 0,4 KiB, `stats.json` 0,6 KiB
czas całkowity (`total`)	211,8 ms

8. Teren z pliku: kaldera z wyłomem

Co sprawdzamy: Dane wejściowe z pliku (mapa wysokości CSV): lawa wypływa tylko przez wyłom w ścianie kaldery.

Polecenie: (podgląd w GUI: wynik `heightmap_kaldera`)

```
.\build\Release\engine.exe --config configs\heightmap_kaldera.json --out
results\heightmap_kaldera
```

Status: ✅ OK (kod 0, 3,1 s z uruchomieniem procesu)

	Wynik
siatka	100 × 100 × 45 = 450 000 komórek, sąsiedztwo moore, brzeg absorbing
kroki CA / koniec	234 kroków, brak płynnej lawy
objętość lawy	1 511 komórek (płynnej na końcu: 0), największy zasięg od krateru: 41,1 komórek
czas CA (`caSim`)	3,04 s (28,9 ns na komórkę i krok)
MC: domena	basalt: 1 511 komórek, Q = 50, 100 MCS, kT = 0.0, mobilność: cooling
MC: ziarna	1 404 → 551; średnie ziarno 1,08 → 2,74 kom., największe 22
MC: energia / akceptacja	5 539 → 4 083; zaakceptowano 27,1 % z 32 621 prób
MC: efekt stygnięcia	wolno stygnące (mobilność ≥ 0.5): średnie ziarno 5,7 kom.; szybko stygnące: 2,6 kom. → **×2,2**
czas MC (`mcSim`)	9,3 ms (284,1 ns na próbę)
pamięć (szczyt)	25,8 MiB zmierzone, model z kodu 20,1 MiB (46,8 B/komórkę)
pliki	`config_used.json` 1,0 KiB, `ca_result.bin` 5,6 MiB, `mc_result.bin` 1,7 MiB, `mc_stats.csv` 0,4 KiB, `stats.json` 5,7 KiB
czas całkowity (`total`)	3,13 s

9. Walidacja configu (błędne dane wejściowe)

Co sprawdzamy: silnik sprawdza config przed liczeniem i zwraca błędy po polsku (tego samego używa formularz GUI, przycisk **Sprawdź**).

Polecenie:

```
.\build\Release\engine.exe --validate --config configs\test_3d.json --set space.nx=-5
```

Status: ✅ błąd wykryty (kod 1 = błąd configu)

```
{
  "ok": false,
  "errors": [
    "Parametr „space.nx” musi być z zakresu 1..100000, podano: -5"
  ],
  "warnings": []
}
```

10. Benchmark: płaska tablica (vector) vs `vector<vector<vector>>` (macierz)

Co sprawdzamy: te same przypadki na obu reprezentacjach siatki: czas samego automatu (mediana) i czy wyniki są identyczne komórka po komórce (kolumna „zgodne”).

Polecenie:

```
.\build\Release\engine.exe --benchmark --quick
```

Status: ✅ wszystkie wyniki zgodne (kod 0), 18,8 s

```
Benchmark: vector vs macierz (mediana z 3 powtórzeń, czas samego CA)
CPU: 13th Gen Intel(R) Core(TM) i9-13900HX, kompilator: MSVC 19.38, build: Release

siatka      komorki  ziarna  kroki  vector [ms]  macierz [ms]  mac/vec  zgodne  tryb
10x10x10    1000     10      9      0.201        0.242         1.20     tak     classic/vn
10x10x10    1000     10      4      0.347        0.441         1.27     tak     classic/moore
25x25x25    15625    62      10     1.21         1.53          1.26     tak     classic/vn
25x25x25    15625    62      6      1.87         2.24          1.20     tak     classic/moore
50x50x50    125000   100     19     18.86        22.80         1.21     tak     classic/vn
100x100     10000    10      39     3.43         5.20          1.52     tak     classic/vn
200x1x100   20000    1       351    100.1        118.3         1.18     tak     lava/moore
64x64x48    196608   3       284    3009.1       3329.3        1.11     tak     lava/moore

Wyniki zapisane w results\benchmark (benchmark.csv, benchmark_meta.json)
```